Recitation 2 Worksheet: a Live Wikipedia Dashboard

Big Data Management · University of Haifa · Lecturer: Artem Barger

9 November 2026 · consolidate at home · not graded

How this worksheet works

In the recitation you tested four claims of Lecture 2 on 20 minutes of the real Wikipedia edit stream: a buggy deploy corrupts counters forever (Act 1); raw events make the same bug a cheap recompute (Act 2); a speed layer covers the batch gap, and the seam between them must not overlap (Act 3); Kappa replaces both by replaying the log (Act 4). This worksheet has two parts:

- **Part A**: the five in-class items, in one table. Redo them if you missed the session.
- **Part B, at home**: five new questions, h1–h5, answered with the *same* programs and one changed setting each. About one hour.

You write what you observed into `observations.py`; `check.py` says `ok` or `WRONG` without revealing the expected value. Nothing is graded or collected. Each Part B item ends with a written question: answer it in 2–4 sentences and bring it to Recitation 3.

Your copy of the code

You need Python 3.8 or newer and git; no packages, no GitHub account.

```
git clone https://github.com/bigdata-management-course/r2-lambda.git
cd r2-lambda
python3 check.py
```

The last two lines must be

```
in class: c1 FAIL  c2 FAIL  c3 FAIL  c4 FAIL  c5 FAIL
at home:  h1 FAIL  h2 FAIL  h3 FAIL  h4 FAIL  h5 FAIL
```

`FAIL` only means “not filled in yet”. On Windows use `py` instead of `python3`.

Rules and facts you need

- The data is `recording.jsonl.gz`: 46,735 events of the Wikimedia stream `recentchange`, 7 November 2026, 22:52–23:12 UTC. Every program replays it (`--speed N` sets how fast; `--live` reads the real stream instead). Acts 1–4 count only edits and new pages of Wikipedia articles: 7,211 of the events.
- Each program has its settings as CAPITALISED names at the top. Change one, run, record, and undo it with `git checkout <file>` before the next item. `check.py` warns if `core.py` or the recording differ from the original.
- In `observations.py` write numbers as numbers (`1803`), a percentage as the number before the % sign (`12.5`). `python3 check.py h1` checks one item, `python3 check.py h1 h2` several.
- `collect.py` (or `dashboard.py`) must have run once so that `data/master/` exists before `batch.py`.

Part A: the in-class items

	run	**record in** `observations.py`
c1	`python3 counter.py`	`V2_ROW`: final count of the broken row `enwiki:`; `FIRST_REAL_COUNT`: final count of the first real page in the list
c2	`python3 collect.py`, then `python3 batch.py`, once more with `KEY = core.page_v2`	`BATCH_PAGES`, `V2_PAGES`: "pages" in the two batch views
c3	`python3 batch.py`	`LATE_EVENTS`: edits more than `LATE = 10` s late; `MOVED`: edits in the file of a later minute
c4	delete `since=T` in `core.py` line 173, then `python3 dashboard.py`	`SEAM_SEEN`, `SEAM_TOTAL`: "edits seen" and "query total" in the frame 23:02:19
c5	`kappa.py` with `KEEP = lambda e: not e["bot"]`	`BOT_SHARE`: dropped, in %; `ENWIKI_KEPT`: the "kept" column of enwiki

Part B: at home

h1. Offsets: where an event sits in the log

at home

Goal. See that every event has a position in Kafka's log, the number Act 4 replays from.

Task. In `tap.py` set `SHOW = 1000`. Run `python3 tap.py --speed 0`. Record `OFFSET_1000`: the offset in the `meta.topic` line of the close-up.

Self-check. `python3 check.py h1`. Undo: `git checkout tap.py`.

Hints. The close-up shows the last event printed, here event 1000.

Written question. Compare the offset of event 20 (in class) with the offset of event 1000. What does the difference tell you about how Kafka numbers the records of one partition?

h2. What the deploy cost one page

at home

Goal. Measure, for one page, the damage of Act 1.

Task. In `counter.py` set `V2 = None`. Run `python3 counter.py --speed 0`. Record `LEON_NO_BUG`: the final count of `frwiki:Léon Grandin`.

Self-check. `python3 check.py h2`. Undo: `git checkout counter.py`.

Hints. The page is in the final list; its last line shows its edits per minute. Compare with the run with `V2 = (6, 9)`.

Written question. With the deploy, which row holds this page's missing edits? Could you, given only the final counter, say how many of that row's edits belong to Léon Grandin?

h3. How late is late

at home

Goal. See how the count of late events depends on the threshold.

Task. In `batch.py` set `LATE = 20`. Run `python3 batch.py`. Record `LATE_20`: the number of late edits.

Self-check. `python3 check.py h3`. Undo: `git checkout batch.py`.

Hints. The first line after the top 10 counts late edits. With `LATE = 10` it was 86.

Written question. The number of edits that "sit in the file of a later minute" does not change with `LATE`. Why not?

h4. A slower batch

at home

Goal. See how the batch horizon moves the split between batch and speed, not the total.

Task. In `dashboard.py` set `BATCH_LAG = 4`. Run `python3 dashboard.py --speed 0`. Record `BATCH_LAG4`: the batch column of the top page in the last frame.

Self-check. `python3 check.py h4`. Undo: `git checkout dashboard.py`.

Hints. `BATCH_LAG` says how many minutes the batch view stays behind the newest closed file. Look at the second line of the last frame: the horizon changed.

Written question. The total column of the top page is the same as with `BATCH_LAG = 2`. Why must it be?

h5. A new question, by replay

at home

Goal. Answer a new question with Kappa: change one line, replay.

Task. In `kappa.py` set `KEEP = lambda e: e["type"] == "new"` (new articles only). Run `python3 kappa.py`. Record `NEW_KEPT`: kept edits in the head line, and `NEW_ENWIKI`: the kept column of enwiki.

Self-check. `python3 check.py h5`. Undo: `git checkout kappa.py`.

Hints. The head line reads "… edits replayed, k kept, …".

Written question. What would the same question cost in the Lambda version of Act 3? Name the parts you would have to change.